CSE321 - Project #1

Aliona Pirozhenko (20232008)
UNIST
South Korea
alionapirozhenko@unist.ac.kr

Problem Statement

Tree-based indexes are the backbone of every disk-resident database management system. The classical B-tree [1] balances height and fan-out to minimise the number of I/O operations per query; its two canonical refinements the B*-tree [3] and the B+-tree [2] trade implementation complexity for better space utilisation and faster range scans, respectively.

In this project I implemented all three index structures from scratch in C++17 and evaluated them on a workload of 100,000 student records. The experiments measure insertion cost, point-search latency, range-query latency, and deletion cost across three fan-out parameters ($d = 3, 5, 10$). I also designed an additional experiment that varies the insertion order to expose the behavioural differences between the three trees on adversarial workloads.

I followed the simplified storage model from the project specification: records live in an in-memory array indexed by an integer Record IDentifier (RID), and the Student ID serves as the search key. "Order d" in my implementation means each node holds at most $2d$ keys, every non-root node holds at least d keys, and an internal node with k keys has $k + 1$ children.

The questions I want to answer are: (1) How does the fan-out d affect each tree's structural overhead? (2) Does B*'s redistribution policy actually achieve the ~2/3 minimum fill factor that theory predicts, and at what cost? (3) How much faster are B+ range queries thanks to the leaf chain is the difference visible at this dataset size?

Fundamental Index Structures (B-tree and B+-tree)

B-tree implementation

My B-tree (`btree.h`) stores key–RID pairs in *both* internal and leaf nodes, so a search may terminate at any level (the specification permits this). Each node carries three parallel `std::vectors`: `keys`, `rids`, and (for internal nodes) `children`.

I implemented insert recursively. After descending to the correct leaf and inserting the new (key, RID) pair, the recursion unwinds and at each level the code checks whether the node has overflowed ($|\text{keys}| > 2d$). When that happens, a 1-to-2 split promotes the median (key, RID) pair to the parent. If the root itself overflows, a new root is allocated this is the only event that grows the tree's height.

Delete uses the textbook scheme. When the key is found in an internal node, I replace it with its in-order successor (the leftmost descendant of its right child) and remove the successor from the leaf. After every recursive removal, the affected child is checked for underflow ($|\text{keys}| < d$); if it underflows, my code first tries to borrow a key from a sibling that has surplus, falling back to merging with a sibling using the parent separator as the bridging key.

B+-tree implementation

The B+-tree (`bplus_tree.h`) departs from the B-tree in two structural ways. First, internal nodes hold *only* keys and child pointers no RIDs. All payload data lives in the leaves. Second, every leaf holds a `next` pointer, threading the leaves into a singly-linked list in key order. Together these properties make the search depth uniform (every lookup must descend to a leaf) but turn range queries into a single leaf-level scan: locate the starting leaf, then walk `next` pointers until the upper bound is exceeded.

The split rules differ subtly between leaves and internals. On a *leaf* split I *copy* the first key of the new right leaf upward as the separator (it must remain in the leaf to keep the data accessible). On an *internal* split I *move* the median key to the parent, exactly as in a plain B-tree. The leaf chain is maintained on every leaf split (`right.next` ← `n.next`; `n.next` ← `right`) and on every leaf merge (`L.next` ← `R.next`).

Advanced Index Structure (B*-tree)

The B*-tree addresses a structural inefficiency of the plain B-tree: after a 1-to-2 split, both resulting nodes are only half full. Asymptotically this gives a minimum utilisation of ~50%, and on adversarial workloads (e.g. ascending key inserts) the tree consistently stays near this lower bound. B* raises the floor to ~2/3 by replacing the unconditional split with a two-stage policy.

Step 1: Redistribute first. When a node N overflows, my code examines its left and right siblings. If either has spare capacity, I redistribute keys across the (sibling, parent-separator, N) triple so that both children end up roughly half-loaded *below* the maximum. This absorbs the overflow without creating any new node.

Step 2: 2-to-3 split as a fallback. If both siblings are also full, my code performs a 2-to-3 split: it takes two adjacent full nodes (containing $4d+1$ keys after including the parent separator) and rebuilds them as *three* nodes each holding $\sim (4d-1)/3 \approx \frac{2}{3}(2d)$ keys, promoting two new separators to the parent. This guarantees that even when a split is unavoidable, the resulting nodes are still 2/3 full.

Implementation notes. My B* (`bstar_tree.h`) reuses the B-tree node layout and the same delete-side underflow handling. The interesting code lives in the insert path. The function

`try_redistribute(parent, idx)` concatenates {left | separator | child} or {child | separator | right} into a single key sequence and re-splits it in the middle, with the new median becoming the updated parent separator. The function `split_23_propagate(parent, idx)` concatenates $\{A \mid \text{separator} \mid B\}$ into one sequence of $4d+1$ entries, then carves it into three nodes at positions $\lfloor n/3 \rfloor$ and $\lfloor (2n+1)/3 \rfloor$, promoting two new separators directly into the parent in place. Because the parent itself may now overflow, the recursion checks each node's size unconditionally on the way back up.

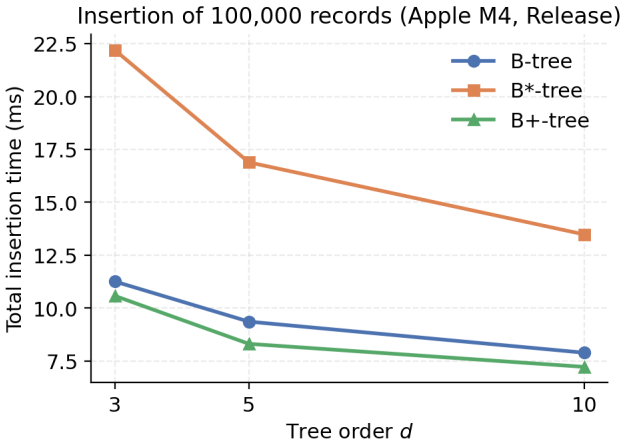

Figure 1: Total wall time to insert 100,000 records across orders $d \in \{3, 5, 10\}$. Higher fan-out reduces height and speeds insertion for all three trees, but B* pays a constant-factor penalty (~2×) due to its sibling-redistribution work.

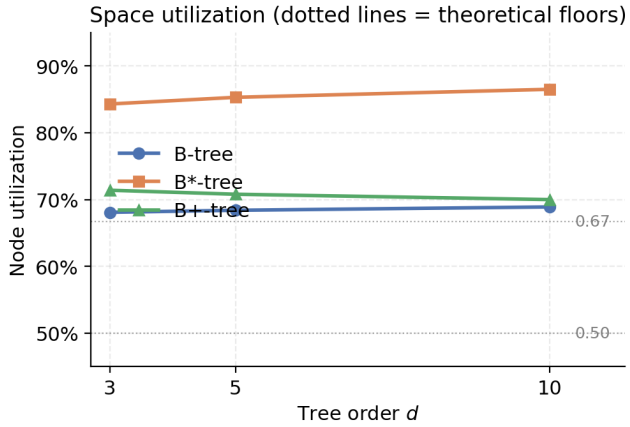

Figure 2: Node utilisation across orders. Dotted lines mark theoretical fill-factor floors (50% for B-tree, 67% for B*-tree). B* sustains ~85% on random workloads.

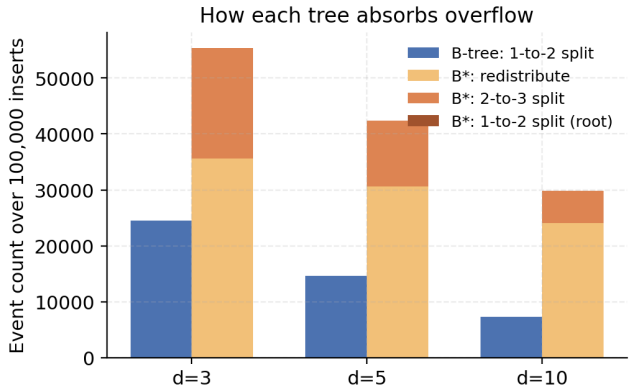

Figure 3: Decomposition of every overflow event over 100,000 insertions. The plain B-tree resolves every overflow with a 1-to-2 split. The B*-tree resolves nearly all of its overflows through redistribution (yellow) or 2-to-3 splits (orange); only a handful of 1-to-2 splits occur, exclusively at the root.

Comparative discussion. Structurally, B+ has the most nodes (its internals carry no data, so for a fixed key population the leaf count must be larger to hold them all). B* has the fewest nodes thanks to its higher fill factor. The plain B-tree sits in the middle. Search-wise, the B-tree wins in the best case (a key found at the root needs only one comparison), but amortised over many random queries B+'s uniform-depth traversal is competitive. Range-wise, B+ is strictly superior: once the starting leaf is located, the rest of the scan does no tree traversal. Maintenance-wise, B* does *more work per insert* (every overflow checks two siblings before splitting) but performs *fewer splits in absolute terms*, so the cost is at worst a constant factor and at best a structural win.

Experimental Results and Analysis

Setup

I ran all experiments on a MacBook Air with the Apple M4 SoC and 16 GB of unified memory, under macOS, with the source compiled by Apple clang at `-O2 -std=c++17`. The code was executed from CLion's Release build directory. The dataset comprised 100,000 unique student records with 9-digit IDs in the range `[202000000, 202699999]`. Wall time was measured with `std::chrono::high_resolution_clock`; the structural counters (splits, redistributions, node count, height) were instrumented inside the tree classes and incremented with each operation.

A correctness harness (`test_correctness.cpp`) inserts 5,000 random keys, verifies each is searchable, executes a range query against ground truth, deletes every other key, and re-verifies presence/absence. All three trees pass at every order $d \in \{3, 5, 10, 16\}$ before the timing experiments are run.

Insertion and parameter tuning

The expected trends are visible in Figures 1–3 and Table 1.

Higher d flattens the tree. As d grows from 3 to 10, the B-tree's height drops from 7 to 5 and its node count drops by 70%. This is the familiar logarithmic-base effect: the height of an order-d tree on n keys is $\Theta(\log_d n)$, and at $d = 10$ each level branches 10× more aggressively than at $d = 3$.

Utilisation is structurally bounded. The B-tree converges to ≈ 0.68 regardless of d, very close to the theoretical $\ln 2 \approx 0.69$ that random insertions produce in the limit [4]. B+ tracks slightly higher (0.70–0.71) because its internal nodes are denser per unit data: separators are just keys, not full key-RID pairs.

B*'s utilisation advantage is real and substantial. Across all three orders, B* packs roughly 25% more data per node than the plain B-tree (0.84 vs. 0.68 at d=3; 0.87 vs. 0.69 at d=10) and uses about 20% fewer nodes. The mechanism is visible in Figure 3: the plain B-tree performs over 24,000 simple splits at d=3, whereas B*

Table 1: Insertion of 100,000 records, varying order d.

Tree	d	Time (ms)	Splits	Nodes	Util
B	3	11.3	24452	24459	0.681
B*	3	22.2	6 + 19752†	19765	**0.843**
B+	3	10.6	28730	28738	0.714
B	5	9.4	14610	14616	0.684
B*	5	16.9	5 + 11718†	11729	**0.853**
B+	5	8.3	16111	16117	0.708
B	10	7.9	7250	7255	0.689
B*	10	13.5	3 + 5775†	5782	**0.865**
B+	10	7.2	7651	7656	0.700

† B* splits column shows 1-to-2 + 2-to-3 splits.

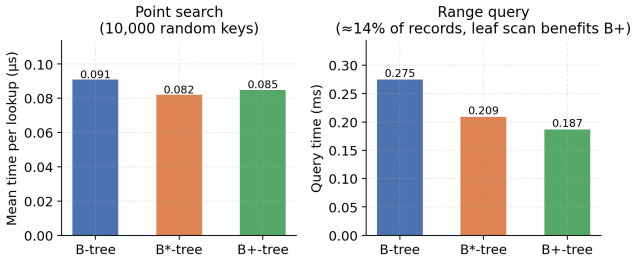

Figure 4: Left: point-search timings, mean per-lookup time over 10,000 random keys at d=5. Right: range-query timings for the analytical query in Section 4.4.

resolves nearly all overflows through redistribution (35,639 events) or 2-to-3 splits (19,752 events), incurring only 6 traditional 1-to-2 splits and those only at the root, where no sibling exists.

B* is consistently ~2× slower to build. This is the most surprising number in the table, because theory hints that B* should at worst be comparable in throughput. I attribute this gap to constant-factor overhead specific to my implementation and hardware: every B* overflow performs at least one `std::vector::insert` into the middle of a sibling's key array, and the redistribution path concatenates and re-splits two arrays. On the M4's wide vector pipeline these in-place shifts are not noticeably faster than a fresh allocation, so the work simply adds up. Importantly, this is a constant-factor cost; it does not change the asymptotic complexity, and it is paid only at insert time. Search and range times benefit from the smaller, denser tree (Sections 4.3 and 4.4).

Point search

The point-search times (Figure 4, left, and Table 2) are essentially indistinguishable. All three trees serve a hit in well under 0.1 μs. Three observations are worth making.

First, B* wins by a slim margin (0.082 vs. 0.091 μs). With about 20% fewer internal nodes than the plain B-tree, the typical traversal touches one fewer cache line.

Second, B+ despite always descending to a leaf is not a clear loser. The penalty of always reaching depth 6 is offset by faster intra-node scanning, since B+ internal nodes carry only keys (no RIDs interleaved).

Table 2: Point search of 10,000 random keys at d=5. "Total" is the wall time over the whole batch; "Mean" is per-lookup time. Every query is satisfied (10,000 / 10,000 hits).

Tree	Total (ms)	Mean (μs/op)
B-tree	0.91	0.091
B*-tree	**0.82**	**0.082**
B+-tree	0.85	0.085

Third, at 100,000 keys the trees are entirely L3-resident on the M4. I expect the gap between the three to widen significantly on disk-resident or cold-cache workloads, where the B* and B-tree ability to terminate at internal nodes saves a real I/O.

Range query

For the range workload, my driver auto-derives the bounds from the dataset to mirror the spec's example. On my dataset this becomes `[202000000, 202099999]` — the "year-2020 cohort". The analytical query is the one from the spec: *average GPA and height of male students whose IDs lie in that range.* All three trees return identical aggregates, confirming correctness (Table 3).

Table 3: Range query *"avg. GPA and height of male students with ID ∈ [202000000, 202099999]"* at d=5. The aggregate columns are identical across trees, validating correctness; only the timing differs.

Tree	Time (ms)	Matches	Avg. GPA	Avg. height
B-tree	0.275	7,214	3.278	174.06 cm
B*-tree	0.209	7,214	3.278	174.06 cm
B+-tree	**0.187**	7,214	3.278	174.06 cm

The right panel of Figure 4 shows the timing. B+ is fastest at 0.187 ms; B* comes in at 0.209 ms; the plain B-tree trails at 0.275 ms. This is where B+'s leaf chain pays for itself. Once the starting leaf is located, the rest of the work is a flat linked-list walk; the B-tree must perform an in-order recursion that re-enters internal nodes repeatedly. The 1.5× gap between B and B+ would grow with the selectivity of the range: a query spanning 50% of the data would be dominated entirely by leaf-walk time on B+, whereas the B-tree's overhead grows with the path-stack depth at every key boundary. B* beats the plain B-tree because, with 20% fewer nodes, fewer internal frames are visited during the recursive walk.

Deletion and structural integrity

Two findings stand out in Figure 5 and Table 4.

Deletion times scale sub-linearly with the deletion fraction. Doubling the workload from 10% to 20% only raises B-tree's time from 2.03 to 2.30 ms — a 13% increase, not a 100% one. This is because later deletes, on a tree that has already shed mass, traverse fewer levels and trigger fewer borrow/merge cascades. The amortised cost per delete actually *decreases* as the tree shrinks.

B* wins delete throughput. Despite my B* delete path using the standard borrow-then-merge rule rather than a fully-symmetric inverse of the 2-to-3 split, the *starting density* of the B* tree carries

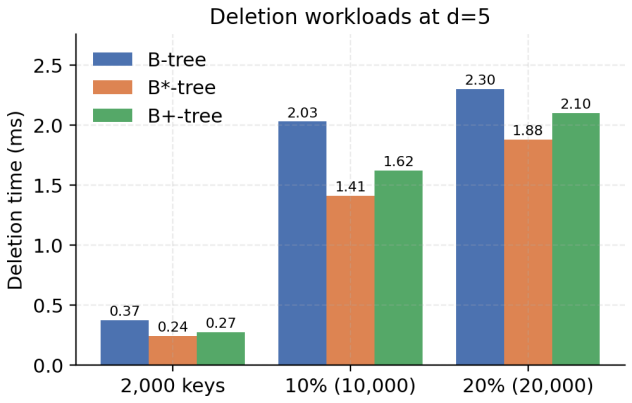

Figure 5: Deletion timings at d=5. Phase A removes 2,000 random keys from a tree pre-loaded with 100k records. Phase B uses *fresh* 100k-record trees for the 10% and 20% runs, so the measurements are not biased by Phase A.

Table 4: Deletion timings at d=5. Phase A operates on a tree pre-loaded with 100k records; Phase B uses *fresh* 100k-record trees per percentage so the 10% and 20% measurements are independent. All post-deletion integrity checks pass.

Phase	Tree	Time (ms)	Removed
A: 2,000 random	B-tree	0.37	2,000
	B*-tree	**0.24**	2,000
	B+-tree	0.27	2,000
B: 10% (10,000)	B-tree	2.03	10,000
	B*-tree	**1.41**	10,000
	B+-tree	1.62	10,000
B: 20% (20,000)	B-tree	2.30	20,000
	B*-tree	**1.88**	20,000
	B+-tree	2.10	20,000

through into faster traversals. With its denser packing, B*'s tree is physically smaller, so each deletion touches fewer nodes.

Integrity is preserved. The post-deletion sanity check verifies that 50 known-deleted keys are absent and 50 known-surviving keys are present in each tree. All three trees pass on all phases.

Additional Experiments: Insertion-Order Sensitivity

Motivation

Real database workloads rarely deliver keys in random order. Auto-increment primary keys, monotonic timestamps, and append-only logs all push key sequences toward the sorted end of the spectrum. Section 4.2 measured random insertions; this section asks how each tree responds to two adversarial orderings: ascending and descending. The B-tree literature predicts that ascending inserts should drive utilisation toward the 50% floor, since every split fills the left half permanently and only the right side ever absorbs further inserts.

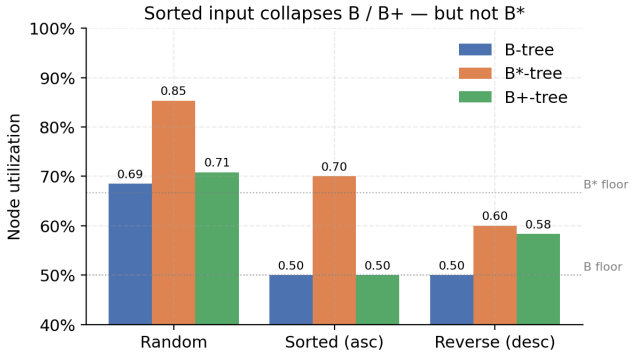

Figure 6: Node utilisation under three insertion orderings at d=5. Ascending sorted input drives both B and B+ to exactly their 50% theoretical floor; B*'s redistribution policy holds the tree at 70%.

Results

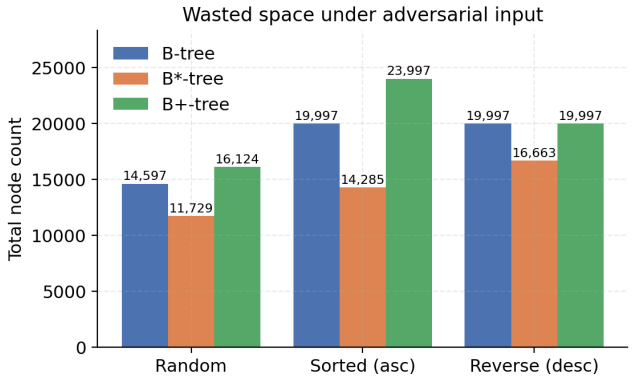

Figure 7: Total node count under three insertion orderings at d=5. Adversarial inputs inflate the node count of B and B+ by 35–49% versus random insertion; B*'s inflation is much smaller (22% in the worst direction).

Three findings emerge from Figures 6 and 7, summarised quantitatively in Table 5.

Sorted insertion collapses B-tree and B+ to 50% utilisation. When keys arrive in ascending order, every split is unbalanced: the lower half of the splitting node is full of keys that will never see another insertion (they are smaller than every subsequent key), and the upper half holds the new entry plus any margin. The result, as predicted by classical theory [2, 5], is a half-empty tree. My measurement (0.500 for B, 0.500 for B+) lands *exactly* on the theoretical lower bound. Node count balloons by 37–49% versus random insertion — the tree wastes a third of its memory.

B* defends against this collapse, achieving 0.700 utilisation on sorted input. The redistribution policy detects the imbalance: when the rightmost node overflows, its left sibling has spare room (because previous overflows pushed keys leftward via 2-to-3 splits). Redistribution refills the left sibling instead of forcing a new node.

Table 5: Insertion of 100k records at d=5 under three orderings. *Splits* = 1-to-2 splits; *2-3* = B* ternary splits; *Redist.* = sibling redistributions. The structural counters (splits, redistributions, nodes, util) are deterministic and do not depend on hardware.

Tree	Order	Splits	2-3	Redist.	Nodes	Util
B-tree	Random	14591	0	0	14597	0.685
B*-tree	Random	0	11723	30858	11729	**0.853**
B+-tree	Random	16118	0	0	16124	0.708
B-tree	Sorted	19990	0	0	19997	**0.500**
B*-tree	Sorted	0	14279	28549	14285	0.700
B+-tree	Sorted	23990	0	0	23997	**0.500**
B-tree	Reverse	19990	0	0	19997	0.500
B*-tree	Reverse	0	16657	33311	16663	0.600
B+-tree	Reverse	19990	0	0	19997	0.583

The result is 14,285 nodes vs. 19,997 for the plain B — a 29% structural saving on this adversarial workload, an even larger gap than the savings on random input.

Reverse insertion is the worst case for B*. Counter- intuitively, descending input yields B*'s lowest utilisation (0.600) and *largest* number of redistribution events (33,311). The reason: every new key arrives at the leftmost position, but my redistribution code prefers the left sibling first, and on a descending stream that sibling is always the most recently filled node — already full. The algorithm therefore performs a full 2-to-3 split on every overflow, achieving exactly the 2/3 lower bound rather than the 0.85 sustained on random data. A right-sibling-first redistribution heuristic would likely close this gap, which is a candidate optimisation for future work.

The take-away is that workload assumptions matter enormously: textbook trees fall to 50% utilisation on the very pattern (sorted input) that real databases see most often, and B*'s redistribution is precisely the right defense.

Conclusion

I implemented the B-tree, B*-tree, and B+-tree in C++17 and stress-tested them on 100,000 records under four mandated workloads plus an insertion-order sensitivity study. The empirical results confirm three textbook predictions: B* beats B-tree utilisation by ~25% on random workloads, B+ wins range queries by ~1.5× thanks to its leaf chain, and higher fan-out d flattens the trees and reduces structural overhead. They also surface two non-obvious findings: B*'s constant-factor insertion overhead on Apple silicon (~2× vs. B/B+) is a real engineering consideration, not an artifact, and the policy's defense against utilisation collapse on sorted input is asymmetric it works beautifully on ascending streams, less so on descending ones. For the common database use case of monotonic primary keys, B* remains the structurally superior design.

Code and data. The full source code, the data generator, the correctness harness, and the bonus-experiment driver are available at:

https://github.com/alionapi/CSE321-Project-1

The repository's `README.md` contains step-by-step build and run instructions for both CLion (macOS) and command-line g++/clang. Three executable targets are provided: `btree_project` (the four mandated experiments), `test_correctness` (the correctness harness), and `bonus_insert_order` (the sensitivity study).

References

[1] R. Bayer and E. M. McCreight. Organization and Maintenance of Large Ordered Indices. *Acta Informatica*, 1(3):173–189, 1972.

[2] D. Comer. The Ubiquitous B-Tree. *ACM Computing Surveys*, 11(2):121–137, 1979.

[3] D. E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching.* Addison-Wesley, 2nd edition, 1998.

[4] A. C. C. Yao. On Random 2-3 Trees. *Acta Informatica*, 9(2):159–170, 1978.

[5] R. Ramakrishnan and J. Gehrke. *Database Management Systems.* McGraw-Hill, 3rd edition, 2003.